Лекции по C++

2 семестр

Дмитрий Величко

Март 2026

Содержание

1 Ключевое слово `throw` **3**

2 Раскрутка стека **3**

3 try / catch **3**

4 Где встречаются исключения в стандартной библиотеке **4**

5 Runtime-ошибки vs Исключения C++ **4**
5.1 Три вида runtime-ошибок на уровне ОС 4
5.2 Пути к `std::terminate()` 5

6 Копирование при броске исключения **5**
6.1 Где хранится объект исключения? 6
6.2 Повторный проброс 6

7 Приведение типов в `catch` **6**
7.1 Наследование в `catch` 7

8 Контейнеры и итераторы **8**
8.1 `std::vector`: реализация изнутри 8
8.1.1 Почему нельзя писать `new T[newcap]` 8
8.1.2 Placement new 8
8.1.3 Почему нельзя `delete[] arr_` 8
8.1.4 Реализация `reserve` 8
8.1.5 Реализация `push_back` 9
8.2 `vector<bool>`: специализация с битовым хранением 9
8.2.1 Прокси-класс `BitReference` 9
8.2.2 Трюк Ильи Мещерина: узнать тип выражения 10

9 Библиотека `<algorithm>` **11**
9.1 Зачем она вообще нужна 11
9.2 `std::sort` 11
9.3 `std::find` и `std::find_if` 11
9.4 `std::count` и `std::count_if` 12

9.5 std::for_each и std::transform . . . 12
9.6 std::copy и std::back_inserter . . . 12
9.7 Идиома erase-remove . . . 12
9.8 std::accumulate из <numeric> . . . 12

10 Дек (std::deque) 14
10.1 Инвалидация указателей и ссылок в vector . . . 14
10.2 Два главных отличия дека от вектора . . . 14
10.3 Внутреннее устройство дека . . . 14

11 Адаптеры контейнеров 14
11.1 std::stack . . . 15

12 Итераторы и их категории 16
12.1 Что такое итератор . . . 16
12.2 Категории итераторов . . . 16
12.3 iterator_traits . . . 16
12.4 Реализация std::distance . . . 17
12.5 Реализация итератора вектора . . . 17

13 Потоковые итераторы 19
13.1 std::istream_iterator . . . 19
13.2 std::ostream_iterator . . . 19
13.3 Итераторы файловых потоков . . . 19

14 Аллокаторы и управление памятью 21
14.1 Зачем аллокатор нужен контейнеру . . . 21
14.2 Устройство стандартного аллокатора . . . 21
14.3 Проблема: list аллоцирует не T . . . 22
14.4 allocator_traits . . . 22
14.4.1 Политики распространения аллокатора . . . 23
14.5 Allocator-aware контейнер . . . 23
14.6 Перегрузка operator new и operator delete . . . 24
14.6.1 Глобальная перегрузка . . . 24
14.6.2 Классовая перегрузка . . . 24
14.6.3 Когда использовать что . . . 25

1 Ключевое слово throw

Рассмотрим функцию деления:

```
#include <stdexcept>

int divide(int a, int b) {
    if (b == 0) {
        throw std::invalid_argument("division by zero");
    }
    return a / b;
}
```

При делении на ноль поведение программы неопределено, поэтому используется ключевое слово `throw`. Бросать можно **любое выражение**, однако принято бросать объекты стандартных исключений из `<stdexcept>`.

`throw` — один из способов выйти из функции наряду с `return`.

2 Раскрутка стека

При выбросе исключения происходит **раскрутка стека** (stack unwinding):

1. Уничтожаются локальные объекты текущей функции (в обратном порядке), вызываются их деструкторы.
2. Управление передаётся вызывающей функции.
3. Процесс повторяется вверх по стеку, пока не будет найден подходящий блок `catch`.

Пример: если `main` вызвал `f1`, та вызвала `f2`, ..., `f10`, и из `f10` брошено исключение, то локальные объекты уничтожаются начиная с `f10` и заканчивая `main`.

3 try / catch

```
#include <iostream>
#include <stdexcept>

int main() {
    try {
        divide(1, 0);
    } catch (const std::exception& e) {
        std::cout << e.what() << "\n";
    }
}
```

- Метод `what()` определён в `std::exception` и возвращает строку с описанием ошибки — ту самую, что была передана при создании объекта исключения.
- Исключение следует ловить **по константной ссылке**, чтобы избежать лишнего копирования и сохранить полиморфизм.

4 Где встречаются исключения в стандартной библиотеке

- `vector::at()` — бросает `std::out_of_range` при выходе за пределы (в отличие от `operator[]`, который имеет неопределённое поведение).
- `operator new` — бросает `std::bad_alloc` при нехватке памяти.
- `dynamic_cast<T&>` — бросает `std::bad_cast` при неудачном приведении.
- `std::terminate()` — вызывается при не пойманном исключении.

5 Runtime-ошибки vs Исключения C++

Это **принципиально разные вещи**.

5.1 Три вида runtime-ошибок на уровне ОС

Сигнал	Типичные причины
`SIGSEGV` (Segfault)	Выход за пределы массива, разыменование `nullptr`, переполнение стека
`SIGFPE` (Floating Point Exception)	Деление целого на ноль, переполнение целых
`SIGABRT` (Aborted)	Вызов `std::abort()`, `std::terminate()`, падение `assert`

Эти ошибки **не являются** исключениями C++ и **не ловятся** через `catch`:

```
int arr[5];
try {
    arr[10] = 42;  // SIGSEGV -- catch
} catch (...) {
    //
}

try {
    int x = 1 / 0;  // SIGFPE --
                                C++
} catch (...) {
    //
}
```

Важно: деление на ноль — это исключение на уровне *процессора*, не языка. Процессор обращается к ОС, которая посылает `SIGFPE`. Это совпадение терминологии, но разные уровни абстракции.

5.2 Пути к std::terminate()

- Не пойманное исключение (uncaught exception)
- Явный вызов std::terminate() или std::abort()
- Упавший assert (он напрямую вызывает abort())
- Проблема ромбовидного наследования при виртуальных деструкторах

С точки зрения ОС программа, завершившаяся из-за не пойманного C++-исключения, получает вердикт SIGABRT.

Также стоит отметить, что std::runtime_error — это *частный случай* std::exception, то есть обычное C++-исключение, не имеющее отношения к SIGFPE/SIGSEGV/SIGABRT.

6 Копирование при броске исключения

```
#include <iostream>

struct A {
    A()          { std::cout << "A()\n"; }
    ~A()         { std::cout << "~A()\n"; }
    A(const A&) { std::cout << "A(copy)\n"; }
};

void f(int x) {
    A a;
    if (x == 0) {
        throw a;  //

    }
}

int main() {
    try {
        f(0);
    } catch (...) {
        std::cout << "caught!\n";
    }
}
```

Порядок вывода:

1. A() — создание a на стеке
2. A(copy) — копирование в динамическую память при throw
3. ~A() — уничтожение a при раскрутке стека
4. caught!
5. ~A() — уничтожение копии из динамической памяти

- `catch(A a)` — ещё одно копирование из дин. памяти на стек.
- `catch(A& a)` — работаем прямо в дин. памяти, без копирования.

6.1 Где хранится объект исключения?

Стандарт не специфицирует точное место хранения. На практике — в **динамической памяти**. Исключение: `std::bad_alloc` — при нехватке памяти нельзя выделить новую, поэтому компилятор заранее резервирует буфер в **статической памяти** (*emergency buffer*).

Исключения дороги по стоимости: выделение динамической памяти происходит в любом случае. Поэтому их не стоит использовать в горячих путях кода.

6.2 Повторный проброс

```
try {
    f(0);
} catch (const A& a) {
    throw;       //                                        (
                                                  )
    // throw a; //                                          ,

}
```

Если исключение не поймано нигде — гарантируется только вызов `std::terminate()`. Деструкторы при этом могут не вызваться.

7 Приведение типов в `catch`

В `catch` **не работают** обычные неявные преобразования и правила перегрузки. Доступны только:

- Добавление `const` / приведение к ссылке
- Приведение типа потомка к типу родителя

```
try {
    throw 1;  // int
} catch (double d) {
    std::cout << "double\n";     //                    : int ->
        double
} catch (long long l) {
    std::cout << "long long\n"; //                    : int ->
        long long
} catch (...) {
    std::cout << "other\n";      //
}
```

7.1 Наследование в catch

Блоки catch проверяются **сверху вниз**, побеждает первый подходящий:

```
struct Mom {};
struct Son : Mom {};

int main() {
    try {
        throw Son{};
    } catch (const Mom& m) {
        std::cout << "Mom\n"; //                    : Son
                                        Mom
    } catch (const Son& s) {
        std::cout << "Son\n"; //
    } catch (...) {
        std::cout << "other\n";
    }
}
```

Вывод: Mom. Именно поэтому обработчики потомков нужно ставить **перед** обработчиками родителей, иначе они никогда не сработают.

8 Контейнеры и итераторы

8.1 std::vector: реализация изнутри

Начнём с устройства `std::vector` и в первую очередь рассмотрим реализацию метода `push_back`.

Структура вектора состоит из трёх полей: указателя на данные `arr_`, размера `sz_` и ёмкости `cap_`.

8.1.1 Почему нельзя писать `new T[newcap]`

При реаллокации наивная реализация могла бы выглядеть так:

```
T* newarr = new T[newcap]; //                    !
```

Это не работает по двум причинам:

- Мы не знаем, есть ли у `T` конструктор по умолчанию.
- Нам нужно выделить *сырую* память и заполнять её по мере надобности, а не создавать все объекты сразу.

8.1.2 Placement new

Решение — выделить сырую память через `char`, а объекты создавать по месту с помощью **placement new**:

```
new (ptr) T(args...);  //                                T
                    ptr
```

Почему нельзя просто написать `newarr[i] = arr_[i]`? Потому что `newarr` указывает на сырые байты — объекта `T` там ещё нет, и присваивать можно только существующему объекту. `memcpy` тоже не подходит: он не работает корректно, если объект содержит указатель или ссылку на своё собственное поле (например, `std::string` с small-string optimization или декартово дерево).

8.1.3 Почему нельзя `delete[] arr_`

При освобождении старой памяти нельзя писать `delete[] arr_`: это вызовет деструктор для каждого из `cap_` слотов, хотя реальных объектов там только `sz_`. Нужно:

1. вручную вызвать деструктор для каждого существующего объекта;
2. освободить сырую память через `delete[] reinterpret_cast<char*>(arr_)`.

8.1.4 Реализация `reserve`

```
template<typename T>
class vector {
    T*     arr_;
    size_t sz_;
    size_t cap_;
```

```
    void reserve(size_t newcap) {
        T* newarr = reinterpret_cast<T*>(new char[newcap *
           sizeof(T)]);
        size_t i = 0;
        try {
            for (; i < sz_; ++i)
                new (newarr + i) T(arr_[i]); // placement new
        } catch (...) {
            for (size_t k = 0; k < i; ++k)
                (newarr + k)->~T();
            delete[] reinterpret_cast<char*>(newarr);
            throw;
        }
        for (size_t k = 0; k < sz_; ++k)
            (arr_ + k)->~T();
        delete[] reinterpret_cast<char*>(arr_);
        arr_ = newarr;
        cap_ = newcap;
    }
```

Обработка исключений в `reserve`:

- Если `new char[...]` бросает `std::bad_alloc` — мы просто не выделили память, ничего не испорчено, вылетаем наверх.
- Если placement new бросает исключение в конструкторе на шаге `i` — нужно уничтожить уже созданные объекты (с 0 по `i-1`), освободить `newarr` и перебросить исключение.

8.1.5 Реализация `push_back`

```
    void push_back(const T& value) {
        if (sz_ == cap_)
            reserve(cap_ > 0 ? cap_ * 2 : 1);
        new (arr_ + sz_) T(value);
        ++sz_;
    }
}; //            class vector
```

8.2 `vector<bool>`: специализация с битовым хранением

`vector<bool>` — это печально известная специализация, которая ломает многие интуитивные ожидания от вектора.

Идея: хранить элементы побитово, упаковывая по 8 штук в `char`. Это экономит память, но делает невозможным взять ссылку `bool&` на отдельный элемент — ссылка не может указывать на отдельный бит.

8.2.1 Прокси-класс `BitReference`

Чтобы сохранить синтаксис `v[i] = true`, вводится прокси-класс:

```cpp
template<>
class vector<bool> {
    char*  arr_;
    size_t sz_;
    size_t cap_;

    struct BitReference {
        char*   cell;
        uint8_t idx;

        BitReference(char* cell, uint8_t idx) : cell(cell), idx(idx) {}

        void operator=(bool b) {
            if (b) *cell |=  (1 << idx);
            else   *cell &= ~(1 << idx);
        }

        operator bool() const {
            return *cell & (1 << idx);
        }
    };

public:
    BitReference operator[](size_t idx) {
        return BitReference(arr_ + idx / 8, idx % 8);
    }
};
```

8.2.2 Трюк Ильи Мещерина: узнать тип выражения

```cpp
template<typename T>
class Debug {
    Debug(T) = delete;
};

int main() {
    std::vector<bool> v(10);
    v[5] = true;
    // Debug d(v[5]); -- : std::__bit_reference
}
```

Почему это проблема: `vector<bool>` нарушает концепцию контейнера — из него нельзя взять настоящую ссылку на элемент (`bool& r = v[5]` не скомпилируется), и многие шаблонные алгоритмы ломаются при работе с ним. Поэтому для хранения битовых флагов часто предпочитают `std::vector<char>` или `std::bitset`.

9 Библиотека <algorithm>

9.1 Зачем она вообще нужна

Стандартная библиотека предоставляет **готовые алгоритмы**, которые работают с любыми контейнерами через **итераторы**. Ключевая идея: алгоритм не знает, что за контейнер перед ним — он получает пару итераторов `[begin, end)` и работает с ними.

```
#include <algorithm>
#include <vector>

int main() {
    int arr[] = {5, 3, 1, 4, 2};
    std::sort(arr, arr + 5);

    std::vector<int> v = {5, 3, 1, 4, 2};
    std::sort(v.begin(), v.end());
}
```

9.2 std::sort

Сортирует диапазон **на месте**. Сложность: $O(n \log n)$ (introsort).

```
std::vector<int> v = {5, 3, 1, 4, 2};
std::sort(v.begin(), v.end()); //

std::sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; //
});
```

Важно: компаратор должен задавать *строгий слабый порядок* (`<`, а не `<=`). Если написать `a >= b` — UB.

9.3 std::find и std::find_if

Линейный поиск. Возвращает итератор на первый подходящий элемент, или `last` если ничего не нашли.

```
std::vector<int> v = {1, 2, 3, 4, 5};

auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) std::cout << *it; // 3

auto it2 = std::find_if(v.begin(), v.end(), [](int x) {
    return x % 2 == 0; //                                -> 2
});
```

`end()` как «не найдено» — универсальное соглашение, потому что алгоритм не знает о типе контейнера и не может вернуть `-1`.

9.4 std::count и std::count_if

```
std::vector<int> v = {1, 2, 2, 3, 2, 4};
int cnt  = std::count(v.begin(), v.end(), 2);
                                // 3
int even = std::count_if(v.begin(), v.end(), [](int x){ return x
    %2==0; }); // 4
```

9.5 std::for_each и std::transform

```
std::vector<int> v = {1, 2, 3, 4, 5};

std::for_each(v.begin(), v.end(), [](int& x) { x *= 2; });
// v == {2, 4, 6, 8, 10}

std::vector<int> out(v.size());
std::transform(v.begin(), v.end(), out.begin(), [](int x) {
    return x * x;
});
// out == {4, 16, 36, 64, 100}
```

9.6 std::copy и std::back_inserter

```
std::vector<int> src = {1, 2, 3, 4, 5};
std::vector<int> evens;

std::copy_if(src.begin(), src.end(), std::back_inserter(evens),
             [](int x) { return x % 2 == 0; });
// evens == {2, 4}
```

std::back_inserter — итератор-адаптер, который при присваивании вызывает push_back. Нужен, когда выходной контейнер изначально пуст.

9.7 Идиома erase-remove

std::remove **не удаляет** элементы — он лишь сдвигает нужные в начало и возвращает итератор на новый логический конец. Реальное удаление делает erase:

```
std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};
v.erase(std::remove(v.begin(), v.end(), 2), v.end());
// v == {1, 3, 4, 5}
```

9.8 std::accumulate из <numeric>

```
#include <numeric>
std::vector<int> v = {1, 2, 3, 4, 5};
int sum     = std::accumulate(v.begin(), v.end(), 0);
                // 15
```

```
int product = std::accumulate(v.begin(), v.end(), 1,
                [](int acc, int x) { return acc * x; });
                                // 120
```

10 Дек (std::deque)

10.1 Инвалидация указателей и ссылок в vector

```
std::vector<int> v(10);
int* p = &v[5];
int& x = v[5];
v.push_back(1);       //                          !
std::cout << *p;      // UB -- pointer invalidation
std::cout << x;       // UB -- reference invalidation
```

При push_back вектор мог реаллоцироваться. Старый кусок памяти нам больше не принадлежит — все указатели и ссылки на элементы, полученные до реаллокации, недействительны.

10.2 Два главных отличия дека от вектора

1. std::deque **не инвалидирует** указатели и ссылки на существующие элементы при добавлении новых.

2. Дек умеет push_front() и pop_front() за $O(1)$.

10.3 Внутреннее устройство дека

Дек хранит **массив указателей на бакеты** фиксированного размера — T** arr_.

- При push_back: идём вправо по бакету; когда бакет заполнен — выделяем новый бакет справа.

- При push_front: идём влево; когда бакет заполнен — выделяем новый бакет слева и кладём элемент с его конца.

- Дек хранит две пары: (номер бакета начала, индекс внутри него) и (номер бакета конца, индекс внутри него).

Почему не инвалидируются указатели? При переполнении внешнего массива реаллоцируется только он — перепривязываются указатели на бакеты. Сами бакеты с элементами остаются на месте.

operator[] **за** $O(1)$**:** зная индекс начала и размер бакета, пересчитываем номер бакета и смещение за константное время.

У дека нет reserve() и shrink_to_fit() — элементы разбросаны по бакетам, и их нельзя реаллоцировать одним куском.

11 Адаптеры контейнеров

std::stack, std::queue и std::priority_queue — это **не контейнеры**, а **адаптеры**: они оборачивают контейнер и предоставляют ограниченный интерфейс поверх него.

```
std::stack<int>                          s;    //
                                 deque
std::stack<int, std::vector<int>>   s2;   //       vector
std::stack<int, std::list<int>>     s3;   //       list
```

11.1 std::stack

```
template<typename T, typename Container = std::deque<T>>
class stack {
    Container c_;
public:
    void   push(const T& val) { c_.push_back(val); }
    void   pop()              { c_.pop_back(); }
    T&     top()              { return c_.back(); }
    bool   empty() const      { return c_.empty(); }
    size_t size()  const      { return c_.size(); }
};
```

Почему `pop()` возвращает `void`? Если бы `pop()` возвращал значение, пришлось бы скопировать элемент перед удалением. Если конструктор копирования бросит исключение — элемент уже удалён и значение потеряно. Разделение на `top()` (посмотреть) и `pop()` (удалить) даёт exception safety. Вернуть ссылку тоже нельзя: после `pop()` она стала бы битой.

12 Итераторы и их категории

12.1 Что такое итератор

Итератор — это тип, который позволяет **обходить последовательность**. Это обобщение указателя: его можно разыменовать (`*it`), инкрементировать (`++it`) и сравнивать (`it != end`). Обычный сырой указатель — полноценный итератор.

Range-based for — синтаксический сахар:

```
for (int x : s) { ... }
//                                    :
for (auto it = s.begin(); it != s.end(); ++it) { ... }
```

Достаточно реализовать `begin()` и `end()`.

12.2 Категории итераторов

Категория	Что умеет	Примеры
`InputIterator`	`*it`, `++it`, `==`, `!=`	потоковые итераторы
`ForwardIterator`	+ гарантия повторного прохода	`unordered_set`, `forward_list`
`BidirectionalIterator`	+ `--it`	`list`, `set`, `map`
`RandomAccessIterator`	+ `it+=n`, `it[n]`, `<`, `>`	`deque`
`ContiguousIterator`	+ гарантия непрерывности памяти	`vector`, `array`, указатель

Разница между `Input` и `Forward`: `InputIterator` не гарантирует, что при повторном проходе увидим те же данные (поток — прочитанное не вернёшь). `ForwardIterator` эту гарантию даёт.

12.3 `iterator_traits`

Для получения информации об итераторе внутри шаблонной функции:

```
template<typename It>
void foo(It begin, It end) {
    typename std::iterator_traits<It>::value_type x = *begin;
    using Cat = typename std::iterator_traits<It>::
        iterator_category;
}
```

Доступные поля: `value_type`, `reference`, `pointer`, `difference_type`, `iterator_category`.

Почему не `auto x = *begin`? Для `vector<bool>` `auto` выведет `BitReference`, а не `bool`. `value_type` вернёт правильный тип.

12.4 Реализация std::distance

```
template<typename It>
typename std::iterator_traits<It>::difference_type
distance(It first, It last) {
    if constexpr (std::is_same_v<
        typename std::iterator_traits<It>::iterator_category,
        std::random_access_iterator_tag>)
    {
        return last - first;  // O(1)
    }
    typename std::iterator_traits<It>::difference_type i = 0;
    for (; first != last; ++first, ++i) {}
    return i;                 // O(n)
}
```

`if constexpr` вычисляется на этапе компиляции: невыбранная ветка не компилируется вообще. Это позволяет писать `last - first` не боясь, что для non-random-access итератора это не скомпилируется.

12.5 Реализация итератора вектора

```
template<typename T>
class Vector {
public:
    class Iterator {
    public:
        using iterator_category = std::
            random_access_iterator_tag;
        using value_type        = T;
        using difference_type   = std::ptrdiff_t;
        using pointer           = T*;
        using reference         = T&;

        explicit Iterator(pointer ptr) : ptr_(ptr) {}

        reference operator*()  const { return *ptr_; }
        pointer   operator->() const { return  ptr_; }

        Iterator& operator++()    { ++ptr_; return *this; }
        Iterator  operator++(int) { Iterator tmp = *this; ++ptr_
            ; return tmp; }
        Iterator& operator--()    { --ptr_; return *this; }
        Iterator  operator--(int) { Iterator tmp = *this; --ptr_
            ; return tmp; }

        Iterator  operator+(difference_type n) const { return
            Iterator(ptr_ + n); }
        Iterator  operator-(difference_type n) const { return
            Iterator(ptr_ - n); }
```

```
        difference_type operator-(const Iterator& o) const { return ptr_ - o.ptr_; }
        Iterator& operator+=(difference_type n) { ptr_ += n; return *this; }
        Iterator& operator-=(difference_type n) { ptr_ -= n; return *this; }
        reference operator[](difference_type n) const { return ptr_[n]; }

        bool operator==(const Iterator& o) const { return ptr_ == o.ptr_; }
        bool operator!=(const Iterator& o) const { return ptr_ != o.ptr_; }
        bool operator< (const Iterator& o) const { return ptr_ < o.ptr_; }
        bool operator> (const Iterator& o) const { return ptr_ > o.ptr_; }
        bool operator<=(const Iterator& o) const { return ptr_ <= o.ptr_; }
        bool operator>=(const Iterator& o) const { return ptr_ >= o.ptr_; }

    private:
        pointer ptr_;
    };

    Vector() : data_(nullptr), size_(0), capacity_(0) {}
    ~Vector() { delete[] data_; }

    void push_back(const T& val) {
        if (size_ == capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
        data_[size_++] = val;
    }

    T&          operator[](std::size_t i) { return data_[i]; }
    std::size_t size()     const { return size_; }
    std::size_t capacity() const { return capacity_; }

    Iterator begin() { return Iterator(data_); }
    Iterator end()   { return Iterator(data_ + size_); }

private:
    void reserve(std::size_t new_cap) {
        T* tmp = new T[new_cap];
        for (std::size_t i = 0; i < size_; ++i) tmp[i] = data_[i];
        delete[] data_;
        data_ = tmp; capacity_ = new_cap;
    }

```

```
    T* data_; std::size_t size_; std::size_t capacity_;
};
```

Пять `type alias`-ов в `Iterator` — контракт с `iterator_traits`. Без них `std::sort` и `std::distance` не смогут узнать категорию итератора.

13 Потоковые итераторы

13.1 std::istream_iterator

```
#include <iterator>
#include <vector>

int main() {
    std::vector<int> v(
        std::istream_iterator<int>(std::cin),
        std::istream_iterator<int>()    //                    -
                        (EOF)
    );
}
```

Это **InputIterator** — одноразовый: прочитанное назад не вернуть. `istream_iterator<int>()` без аргументов — итератор конца потока.

```
std::istream_iterator<int> in(std::cin), eof;
int sum = std::accumulate(in, eof, 0); //
                        stdin
```

13.2 std::ostream_iterator

```
std::vector<int> v = {1, 2, 3, 4, 5};

std::copy(v.begin(), v.end(),
          std::ostream_iterator<int>(std::cout, " "));
//            : 1 2 3 4 5

std::transform(v.begin(), v.end(),
                std::ostream_iterator<int>(std::cout, "\n"),
                [](int x) { return x * x; });
```

`ostream_iterator` — **OutputIterator**: присваивание `*it = val` вызывает `os « val « delim`.

13.3 Итераторы файловых потоков

```
#include <fstream>
#include <iterator>
#include <algorithm>

```

```
int main() {
    std::ifstream fin("input.txt");
    std::ofstream fout("output.txt");

    std::vector<int> v(std::istream_iterator<int>(fin),
                       std::istream_iterator<int>());
    std::sort(v.begin(), v.end());
    std::copy(v.begin(), v.end(),
              std::ostream_iterator<int>(fout, "\n"));
}
```

Алгоритмы не знают, откуда данные читаются и куда пишутся — они работают с абстракцией итератора. В этом и есть смысл всей иерархии.

14 Аллокаторы и управление памятью

14.1 Зачем аллокатор нужен контейнеру

У каждого контейнера есть шаблонный параметр аллокатора:

```
std::list<int>     // = std::list<int, std::allocator<int>>
std::vector<int>   // = std::vector<int, std::allocator<int>>
```

Аллокатор — это **промежуточный класс между контейнером и оператором** `new`. Стек вызовов:

$$\texttt{Container} \to \texttt{Allocator} \to \texttt{operator new} \to \texttt{malloc} \to \texttt{OS}$$

14.2 Устройство стандартного аллокатора

У аллокатора четыре метода:

```
template<typename T>
struct allocator {
    using value_type = T;

    T* allocate(std::size_t n) {
        if (n == 0) return nullptr;
        return static_cast<T*>(::operator new(n * sizeof(T)));
    }

    void deallocate(T* p, std::size_t) {
        ::operator delete(p);
    }

    template<typename... Args>
    void construct(T* p, Args&&... args) {
        ::new (static_cast<void*>(p)) T(std::forward<Args>(args)
            ...);
    }

    void destroy(T* p) {
        p->~T();
    }
};
```

- `allocate` — выделяет сырую память, конструкторы не вызываются.
- `deallocate` — освобождает память, деструкторы не вызываются.
- `construct` — вызывает конструктор по адресу (placement new).
- `destroy` — явно вызывает деструктор, память не освобождает.

14.3 Проблема: list аллоцирует не T

std::list<int> хранит не просто int, а узлы:

```
template<typename T>
struct ListNode { T value; ListNode* prev; ListNode* next; };
```

Но аллокатор шаблонизирован по T = int. Решение комитета — **rebind**: вложенный шаблон в аллокаторе, который перепривязывает его к другому типу:

```
template<typename T>
struct allocator {
    template<typename U>
    struct rebind { using other = allocator<U>; };
};

// list                          :
using NodeAlloc = typename Alloc::template rebind<ListNode<T>>::
    other;
NodeAlloc node_alloc_;  //                                    ,
                    T
```

14.4 allocator_traits

std::allocator_traits<Alloc> — адаптер с единым интерфейсом к любому аллокатору и разумными дефолтами для отсутствующих методов.

```
template<typename Alloc>
struct allocator_traits {
    using value_type      = typename Alloc::value_type;
    using pointer         = value_type*;
    using size_type       = std::size_t;
    using difference_type = std::ptrdiff_t;

    template<typename U>
    using rebind_alloc = typename Alloc::template rebind<U>::
        other;

    static pointer allocate(Alloc& a, size_type n)
        { return a.allocate(n); }
    static void deallocate(Alloc& a, pointer p, size_type n)
        { a.deallocate(p, n); }

    //              Alloc        construct -- placement new

    template<typename T, typename... Args>
    static void construct(Alloc&, T* p, Args&&... args)
        { ::new (static_cast<void*>(p)) T(std::forward<Args>(
            args)...); }

    //              Alloc        destroy --

    template<typename T>
```

```
    static void destroy(Alloc&, T* p) { p->~T(); }

    //

    using propagate_on_container_copy_assignment = std::
        false_type;
    using propagate_on_container_move_assignment = std::
        false_type;
    using propagate_on_container_swap            = std::
        false_type;
};
```

Главная идея: контейнер должен работать с аллокатором *только через* `allocator_traits`. Тогда кастомный аллокатор может не реализовывать `construct`/`destroy` и всё равно работать.

14.4.1 Политики распространения аллокатора

Флаг	Что означает
`propagate_on_container_copy_assignment`	Копировать аллокатор при `operator=` копирования?
`propagate_on_container_move_assignment`	Копировать аллокатор при `operator=` перемещения?
`propagate_on_container_swap`	Менять аллокаторы местами при `swap`?

По умолчанию все три — `false_type`. Это важно для stateful аллокаторов: если два вектора используют разные арены, при свопе нужно либо перетащить элементы, либо поменять аллокаторы.

14.5 Allocator-aware контейнер

```
template<typename T, typename Alloc = std::allocator<T>>
class vector {
    using AllocTraits = std::allocator_traits<Alloc>;

    Alloc  alloc_;
    T*     arr_;
    size_t sz_;
    size_t cap_;

public:
    explicit vector(const Alloc& a = Alloc())
        : alloc_(a), arr_(nullptr), sz_(0), cap_(0) {}

    void push_back(const T& val) {
        if (sz_ == cap_) reserve(cap_ ? cap_ * 2 : 1);
        AllocTraits::construct(alloc_, arr_ + sz_, val);
        ++sz_;
```

```
    }

    ~vector() {
        for (size_t i = 0; i < sz_; ++i)
            AllocTraits::destroy(alloc_, arr_ + i);
        AllocTraits::deallocate(alloc_, arr_, cap_);
    }

private:
    void reserve(size_t newcap) {
        T* newarr = AllocTraits::allocate(alloc_, newcap);
        size_t i = 0;
        try {
            for (; i < sz_; ++i)
                AllocTraits::construct(alloc_, newarr + i, arr_[
                    i]);
        } catch (...) {
            for (size_t k = 0; k < i; ++k)
                AllocTraits::destroy(alloc_, newarr + k);
            AllocTraits::deallocate(alloc_, newarr, newcap);
            throw;
        }
        for (size_t k = 0; k < sz_; ++k)
            AllocTraits::destroy(alloc_, arr_ + k);
        AllocTraits::deallocate(alloc_, arr_, cap_);
        arr_ = newarr; cap_ = newcap;
    }
};
```

Stateless аллокатор (как `std::allocator`) не занимает места благодаря **EBO** (Empty Base Optimization).

14.6 Перегрузка `operator new` и `operator delete`

14.6.1 Глобальная перегрузка

```
void* operator new(std::size_t size) {
    void* ptr = std::malloc(size);
    if (!ptr) throw std::bad_alloc{};
    return ptr;
}
void operator delete(void* ptr) noexcept { std::free(ptr); }

// C++14: sized delete
void operator delete(void* ptr, std::size_t size) noexcept { std
    ::free(ptr); }
```

Затрагивает весь процесс. Используется для профилировщиков памяти и детекторов утечек.

14.6.2 Классовая перегрузка

```
struct Node {
    int value; Node* next;

    void* operator new(std::size_t size) {
        return ::operator new(size); //

    }
    void operator delete(void* ptr) noexcept {
        ::operator delete(ptr);
    }
};
```

При `new Node` компилятор сначала ищет `operator new` в классе, и только если не нашёл — берёт глобальный.

14.6.3 Когда использовать что

Инструмент	Когда применять
Кастомный аллокатор	Своя стратегия памяти для конкретного контейнера
Классовая перегрузка `new/delete`	Свой пул для конкретного класса (например, узлы дерева)
Глобальная перегрузка `new/delete`	Профилирование, детектирование утечек для всей программы